

Operator Overloading and Type Conversion

Operators Overloading

- Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type.
- The advantage of Operators overloading is to perform different operations on the same operand.

Operators Overloading

- Operator that cannot be overloaded are as follows:
 - Class members access operator (. , .*)
 - Scope resolution operator (::)
 - Size operator(sizeof)
 - Condition operator (? :)
- we can't change its syntax, the grammatical rules that govern its use such as the no of operands precedence and associativity. For example the multiplication operator will enjoy higher precedence than the addition operator.

Defining Operators Overloading

- To define an additional task to an operator, we must specify a special function called operator function, which describes the task.
- Syntax:-

```
return-type class-name :: operator op( arg-list)
{
    function body
}
```
- Where return type is the type of value returned by the specified operation and op is the operator being overloaded. The op is preceded by the keyword operator, operator op is the function name.

Rules for Operator Overloading

- Existing operators can only be overloaded.
- The overloaded operator contains at least one operand of the user-defined data type.
- We should declare the operator function operator op() in the public part of the class.
- It may be either a member function or friend function.
- When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
- When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

Operator Function Call

- Overloaded operator functions can be invoked by expressions such as:
 - for unary operators
op x or **x op**;
 - for binary operators
x op y;
 - for unary operator using friend function
operator op(x);
 - for binary operator using friend function.
operator op(x , y);

Overloading Unary Operator

```
#include<iostream>
using namespace std;
class abc
{
    int m,n;
public:
    abc()
    {
        m=8;
        n=9;
    }
    void show()
    {
        cout<<"M:"<<m<<endl;
        cout<<"N:"<<n<<endl;
    }
};

void operator -- ()
{
    --m;
    --n;
};

int main()
{
    abc A;
    cout<<"Given Value:\n";
    A.show();
    --A;
    cout<<"After --A:\n";
    A.show();
    return 0;
}
```

Using Friend Function

```
#include<iostream>
using namespace std;
class abc
{
    int m,n;
public:
    abc()
    {
        m=8;
        n=9;
    }
    void show()
    {
        cout<<"M:"<<m<<endl;

        cout<<"N:"<<n<<endl;
    }
}
```

```
friend void operator -- (abc&);
};
void operator -- (abc &X)
{
    --X.m;
    --X.n;
}
int main()
{
    abc A;
    cout<<"Given Value:\n";
    A.show();
    --A; //operator--(A);
    cout<<"After --A:\n";
    A.show();
    return 0;
}
```

Overloading Unary Minus Operator

```
#include<iostream>
using namespace std;
class Number
{
    int m,n;
public:
    void getdata(int x,int y)
    {
        m=x;
        n=y;
    }
    void show()
    {
        cout<<"M:"<<m<<endl;

        cout<<"N:"<<n<<endl;
    }
}

operator - ()
{
    m=-m;
    n=-n;
};

int main()
{
    Number N;
    N.getdata(-9,5);
    cout<<"Given Value:\n";
    N.show();
    -N;
    cout<<"After Sign Change -N:\n";
    N.show();
    return 0;
}
```

Using Friend Function

```
#include<iostream>
using namespace std;
class Number
{
    int m,n;
public:
    void getdata(int x,int y)
    {
        m=x;
        n=y;
    }
    void show()
    {
        cout<<"M:"<<m<<endl;

        cout<<"N:"<<n<<endl;
    }
friend void operator - (Number&);
}

void operator - (Number &N1)
{
    N1.m=-N1.m;
    N1.n=-N1.n;
}

int main()
{
    Number N;
    N.getdata(-9,5);
    cout<<"Given Value:\n";
    N.show();
    -N; //operator-(N);
    cout<<"After Sign Change -N:\n";
    N.show();
    return 0;
}
```

Postfix/Prefix Increment

```
#include <iostream>
using namespace std;

class Count {
private:
    int value;

public:
    Count()
    {
        value=5;
    }
    // Overload ++ when used as prefix
    void operator ++ ()
    {
        ++value;
    }
    // Overload ++ when used as postfix
    void operator ++ (int)
    {
        ++value;
    }
    void display() {
        cout << "Count: " << value << endl;
    }
};

int main() {
    Count C1;
    C1++;
    cout<<"Postfix Increment:\n";
    C1.display();
    ++ C1;
    cout<<"Prefix Increment:\n";
    C1.display();
    return 0;
}
```

Binary Operator Overloading

```
#include<iostream>
using namespace std;
class complex
{
    float real,img;
public:
    complex()
    {
        real=0;
        img=0;
    }
    complex(float r,float i)
    {
        real=r;
        img=i;
    }
    void show()
    {
        cout<<real<<"+"i"<<img<<endl;
    }
};

complex operator+(complex &p)
{
    complex w;
    w.real=real+p.real;
    w.img=img+p.img;
    return w;
};

int main()
{
    complex C1(3,4);
    complex C2(4,5);
    complex C3;
    C3=C1+C2;
    cout<<"Given Complex Numbers:\n";
    C1.show();
    C2.show();
    cout<<"Sum of These Numbers:\n";
    C3.show();
    return 0;
}
```

Binary Operator Overloading using friend function

```
#include<iostream>
using namespace std;
class complex
{
    float real,img;
public:
    complex()
    {
        real=0;
        img=0;
    }
    complex(float r,float i)
    {
        real=r;
        img=i;
    }
    void show()
    {
        cout<<real<<"+"i"<<img<<endl;
    }
};

friend complex operator+(complex&,complex&);

complex operator+(complex &p, complex &q)
{
    complex w;
    w.real=q.real+p.real;
    w.img=q.img+p.img;
    return w;
}

int main()
{
    complex C1(3,4);
    complex C2(4,5);
    complex C3;
    C3=C1+C2;//C=Operator+(C1,C2);
    cout<<"Given Complex Numbers:\n";
    C1.show();
    C2.show();
    cout<<"Sum of These Tumbers:\n";
    C3.show();
    return 0;
}
```

Type Conversions

- In a mixed expression constants and variables are of different data types. The assignment operations causes automatic type conversion between the operand as per certain rules.
- The type of data to the right of an assignment operator is automatically converted to the data type of variable on the left.
- Consider the following example:
 int x;
 float y = 20.123;
 x=y ;
- This converts float variable y to an integer before its value assigned to x. The type conversion is automatic as far as data types involved are built in types. What if one of the operand is basic type and another is class type? What if they belongs to different classes?

- Special conversion routine is needed for conversion between user defined data items and basic data type, because compiler need to understand the structure and memory requirement of user defined data type before conversion.
- There are three types of situations that arise where data conversion are between incompatible types.
 1. Conversion from built in type to class type.
 2. Conversion from class type to built in type.
 3. Conversion from one class type to another.

Conversion from built in type to class type

- Conversion from basic data type to class type can be performed by using a single parameter in a constructor.

- Example:

```
class Time
{
    int hours;
    int minutes;
    public:
    Time (int t) // constructor
    {
        hours = t / 60; //t is inputted in minutes
        minutes = t % 60;
    }
};

Int x=205;
Time T=x; //total Minutes converted to hour and minutes
```

Program to convert quantity of weight into kilograms and grams xample:

```
#include<iostream>
class weight
{
    int kg, gm;
    public:
    weight ()
    {
        kg = gm = 0;
    }
    weight ( int w )
    {
        kg = w / 1000;
        gm = w % 1000;
    }
    void display ( )
    {
        cout<<"\n kg = "<< kg<<"\t gm = "<<gm ;
    }
};

int main ( )
{
    int wgm;
    cout<<"\n enter weight in grams : ";
    cin>> wgm;
    weight mangoes ;
    mangoes = wgm;
    mangoes.display ( );
    return 0;
}
```

Class to Basic Type

- The constructor functions do not support conversion from a class to basic type. C++ allows us to define a overloaded casting operator that convert a class type data to basic type. The general form of an overloaded casting operator function, also referred to as a conversion function, is:

```
operator type_name()
{
    //Conversion Routine
}

operator int ()
{
    int w ;
    w = (kg * 1000) + gm;
    return ( w );
}
```

The casting operator should satisfy the following conditions.

- It must be a class member.
- It must not specify a return type but must return a value.
- It must not have any arguments.
- Being a member function, it is invoked by an object and values of that object are used inside the function. Hence no argument is passed to conversion function.

Program to convert weight in kilograms and grams to grams only (class to basic type)

```
#include<iostream>
class weight
{
    int kg, gm;
    public:
    weight ()
    {
        kg = gm = 0;
    }
    weight ( int k, int g )
    {
        kg = k ;
        gm = g ;
    }
    operator int ( )
    {
        int w ;
        w = (kg * 1000) + gm;
        return ( w );
    }
    void display ( )
    {
        cout<<"\n weight in kg & gm : "<< kg<<"
        kg"<<gm << " gm ";
    }
};

int main ( )
{
    int wfruits , wvegies ;
    weight fruits(3 , 500 );
    weight vegies( 2 , 250 );
    fruits.display ( );
    vegies.display ( );

    wfruits = fruits ;
    wvegies = vegies;
    int total_weight = wfruits + wvegies;

    cout<<"\n total weight of fruits and
    vegetables : "<<total_weight;
    return 0;
}
```

One Class Type to Another Class Type Conversion

- The conversion between objects of different classes can be done using either a one-argument constructor or a casting operator function. The choice depends upon whether the conversion routine has to be declared in the source class or in the destination class.
- For Example
 C1=N1;
- In the above statement, object N1 of type **Number** is converted to type **Complex** and assigned to C1. Therefore, N1 is the source and C1 is the destination.
- If class **Number** handles the conversion, it will hold a casting operator function. On the other hand, if class **Complex** carries out the conversion, it will do that through a constructor that takes an argument of type class Number.

One Class to Another Class type conversion

- Note that only one of the two conversion routines should be specified; the compiler will yield an error if both are specified since it will not be able to figure out which routine to call.
- Example:

Complex(Number); //Constructor in Destination class

Operator Complex(); //Casting function in source class

Conversion	Conversion takes place in	
	Source class	Destination class
Basic to class	Not applicable	Constructor
Class to Basic	Casting operator	Not applicable
Class to class	Casting operator	Constructor

Example:

```
#include <iostream>
using namespace std;
class stock2 ;
class stock1
{
    int code , item ;
    float price ;
public :
    stock1 ( int a , int b , floatc )
    {
        code = a ;
        item = b ;
        price = c ;
    }
    void disp ()
    {
        cout << " code " << code << " \n " ;
        cout << " items " << item << " \n " ;
        cout << " price per item Rs. " <<
            price << " \n " ;
    }
    int getcode ()
    { return code; }
    int getitem ()
    { return item ; }
    int getprice ()
```

```
    { return price ; }
    operator float ()
    {
        return ( item*price ) ;
    }
};
class stock2
{
    int code ;
    float val ;
public :
    stock2 ()
    {
        code = 0;
        val = 0 ;
    }
    stock2( int x , float y )
    {
        code = x ;
        val = y ;
    }
    void disp ()
    {
        cout << " code " << code << " \n " ;
        cout << " total value Rs. " << val
```

```
<< " \n " ;
    }
    stock2( stock1 p )
    {
        code = p.getcode() ;
        val = p.getitem() * p.getprice() ;
    }
};
int main()
{
    stock1 i1 ( 101 , 10 ,125.0 ) ;
    stock2 i2 ;
    float tot_val = i1;
    i2 = i1 ;
    cout << " Stock Details : Stock 1 type
        " << " \n " ;
    i1.disp ();
    cout << " Stock Value " << " - " ;
    cout << tot_val << " \n " ;
    cout << " Stock Details : Stock 2 type
        " << " \n " ;
    i2.disp () ;
    return 0 ;
```

Example: Miles to Kilometers

```
#include <iostream>
using namespace std;
class Kilometers
{
    double km;
public:
    Kilometers(double k)
    {
        km=k;
    }
    void display()
    {
        cout << km << " kilometers";
    }
    double getValue()
    {
        return km;
    }
};

class Miles
{
    double mi;
public:
    Miles(double m)
    {
        mi=m;
    }
    void display()
    {
        cout << mi << " miles";
    }
    operator Kilometers()
    {
        return
            Kilometers(mi*1.609344);
    }
    Miles(Kilometers kilo)
    {
        mi = kilo.getValue()/1.609344;
    }
};

int main( )
{
    //conversion function(Casting Operator)
    double XX;
    cout<<"Enter Distance in Miles:";
    cin>>XX;
    Miles m1(XX);
    Kilometers k1 = m1;
    m1.display();

    cout << " = ";
    k1.display();
    cout << endl;
    //using the constructor
    cout<<"\n\nEnter Distance in
        Kilometers:";
    cin>>XX;
    Kilometers k2 =XX;
    Miles m2 = k2; // same as:
        //Miles m2 = Miles(k2);
    k2.display();
    cout << " = ";
    m2.display();
    cout << endl;
}
```